



**Department of Electronic and Telecommunication
Engineering**

University of Moratuwa

B.Sc. Engineering – Semester 5

EN3160 – Image Processing and Machine Vision

**Intensity Transformations &
Neighborhood Filtering**

Laboratory: Analog Lab

Group number: 6

Members: 230449N – Nuwanaka W. A. S.

September 12, 2026

Contents

1	Question 1: Piecewise Linear Intensity Transformation	3
1.1	Task Description	3
1.2	Requirements	3
1.3	Implementation Code Stub	3
1.4	Discussion & Observations	4
2	Question 2: Tissue Accentuation in Brain Proton Density Image	5
2.1	Task Description	5
2.2	Requirements	5
2.3	Implementation Code Stub	5
2.4	Discussion & Observations	5
3	Question 3: Gamma Correction in $L^*a^*b^*$ Color Space	6
3.1	Task Description	6
3.2	Mathematical Formulation	6
3.3	Implementation Code Stub	6
3.4	Discussion & Observations	6
4	Question 4: Vibrance Enhancement in HSV Color Space	7
4.1	Task Description	7
4.2	Requirements	7
4.3	Implementation Code Stub	7
4.4	Discussion & Observations	7
5	Question 5: Custom Histogram Equalization	8
5.1	Task Description	8
5.2	Requirements	8
5.3	Implementation Code Stub	8
5.4	Discussion & Observations	8
6	Question 6: Selective Foreground Histogram Equalization	9
6.1	Task Description	9
6.2	Steps	9
6.3	Implementation Code Stub	9
6.4	Discussion & Observations	9
7	Question 7: Sobel Filtering & Separable Filtering	10
7.1	Task Description	10
7.2	Implementation Code Stub	10
7.3	Discussion & Observations	10
8	Question 8: Image Zooming (Nearest-Neighbor & Bilinear)	11
8.1	Task Description	11
8.2	Requirements	11
8.3	Implementation Code Stub	11
8.4	Discussion & Observations	11

9	Question 9: GrabCut Segmentation & Background Blurring	12
9.1	Task Description	12
9.2	Implementation Code Stub	12
9.3	Discussion & Answer to Part (c)	12
10	Question 10: Edge-Preserving Bilateral Filtering	13
10.1	Task Description	13
10.2	Implementation Code Stub	13
10.3	Discussion & Observations	13

1 Question 1: Piecewise Linear Intensity Transformation

1.1 Task Description

Implement a piecewise linear intensity transformation function `intensity_transform(im, breakpoints)` where `breakpoints` defines a mapping from input intensities x to output intensities y , e.g.,

$$\text{breakpoints} = \begin{bmatrix} 0 & 0 \\ 50 & 50 \\ 100 & 150 \\ 150 & 255 \\ 255 & 255 \end{bmatrix} \quad (1)$$

1.2 Requirements

1. Implement `intensity_transform(im, breakpoints)` using lookup tables (LUT) or piecewise linear interpolation (`np.interp`).
2. Load target image `im01.png` (or `im01small.png`).
3. Experiment with different breakpoints to achieve contrast enhancement.
4. Display the transformation curve plot alongside the original and transformed images side-by-side.

1.3 Implementation Code Stub

```
1 import cv2 as cv
2 import numpy as np
3
4 def intensity_transform(im: np.ndarray, breakpoints: np.ndarray) -> np.
  ndarray:
5     """
6     Applies piecewise linear intensity transformation to an input image.
7
8     Parameters:
9         im (np.ndarray): Input image (grayscale or color)
10        breakpoints (np.ndarray): Nx2 matrix of [input_intensity,
  output_intensity]
11
12    Returns:
13        np.ndarray: Transformed image of same shape and uint8 dtype
14    """
15    # TODO: Extract x and y coordinates from breakpoints
16    # TODO: Create Look-Up Table (LUT) for intensity mapping [0..255]
17    # TODO: Apply cv.LUT or np.interp to transform image intensities
18    pass
```

Listing 1: Piecewise Linear Intensity Transformation Stub

1.4 Discussion & Observations

Discuss how changing breakpoints alters image contrast, highlights, and shadow retention...

2 Question 2: Tissue Accentuation in Brain Proton Density Image

2.1 Task Description

Apply piecewise linear intensity transformations on the brain proton density slice (`im02.png` / `im02small.png`) to selectively accentuate:

- (a) White matter
- (b) Gray matter

2.2 Requirements

1. Analyze histogram of the brain proton density slice to identify intensity ranges corresponding to white matter and gray matter.
2. Design custom breakpoint curves for (a) White Matter accentuation and (b) Gray Matter accentuation.
3. Plot transformation curves and display the original vs accentuated images.

2.3 Implementation Code Stub

```
1 # TODO: Load brain proton density slice 'im02.png' or 'im02small.png'
2 # TODO: Define breakpoint arrays for white matter and gray matter
   enhancement
3 # TODO: Call intensity_transform() for both cases
4 # TODO: Plot transformation curves and display original, white matter,
   and gray matter images
```

Listing 2: Tissue Accentuation Code Stub

2.4 Discussion & Observations

Discuss intensity range selections for white and gray matter and visual segmentations achieved...

3 Question 3: Gamma Correction in $L^*a^*b^*$ Color Space

3.1 Task Description

Consider the image shown in Fig. 3 (im03.png / im03small.png).

1. (a) Convert image to $L^*a^*b^*$ color space and apply gamma correction to the L^* plane. State chosen γ value.
2. (b) Display original vs gamma-corrected image, along with histograms of the L^* plane before and after correction.

3.2 Mathematical Formulation

$$L_{\text{out}} = 255 \times \left(\frac{L_{\text{in}}}{255} \right)^\gamma \quad (2)$$

3.3 Implementation Code Stub

```
1 def gamma_correct_L_plane(im_bgr: np.ndarray, gamma: float) -> tuple[np.  
   ndarray, np.ndarray, np.ndarray]:  
2     """  
3     Applies gamma correction to the L* plane in L*a*b* color space.  
4     """  
5     # TODO: Convert BGR to Lab space using cv.cvtColor(im_bgr, cv.  
   COLOR_BGR2LAB)  
6     # TODO: Extract L, a, b planes  
7     # TODO: Compute normalized power law transform: L_out = 255 * ((L_in  
   / 255) ** gamma)  
8     # TODO: Merge L_out, a, b and convert back to BGR space  
9     pass
```

Listing 3: Gamma Correction Code Stub

3.4 Discussion & Observations

Explain the choice of gamma value, impact on shadows/highlights in Lab space vs RGB space...

4 Question 4: Vibrance Enhancement in HSV Color Space

4.1 Task Description

Enhance photograph vibrance on image `im04.jpg` (`im04small.jpg`) by applying the intensity transformation function:

$$f(x) = \min\left(x + a \times 128e^{-\frac{(x-128)^2}{2\sigma^2}}, 255\right) \quad (3)$$

to the Saturation (S) plane in HSV color space, where $x \in [0, 255]$, $a \in [0, 1]$, and $\sigma = 70$.

4.2 Requirements

1. (a) Split `im04.jpg` into Hue (H), Saturation (S), and Value (V) planes.
2. (b) Apply the vibrance transform formula to the S plane.
3. (c) Adjust a for optimal visual results and report chosen a .
4. (d) Recombine $H, S_{\text{enhanced}}, V$ back into BGR image.
5. (e) Display original image, vibrance-enhanced image, and transformation plot $f(x)$ vs x .

4.3 Implementation Code Stub

```
1 def vibrance_transform(S_plane: np.ndarray, a: float, sigma: float =  
    70.0) -> np.ndarray:  
2     """Applies vibrance enhancement function to Saturation channel."""  
3     # TODO: Compute transformation  $f(x) = \min(x + a * 128 * \exp(-((x$   
     $-128)^2)/(2 * \sigma^2)), 255)$   
4     # TODO: Cast back to uint8  
5     pass
```

Listing 4: Vibrance Enhancement Code Stub

4.4 Discussion & Observations

Report the chosen 'a' parameter and explain why Gaussian-weighted saturation boosting avoids clipping vibrant colors...

5 Question 5: Custom Histogram Equalization

5.1 Task Description

Write a custom function from scratch to carry out histogram equalization on `im05.jpg`.

5.2 Requirements

1. Implement `custom_histogram_equalization(img_gray)` without using OpenCV's `cv.equalizeHist`.
2. Compute intensity frequency histogram $h(r_k)$.
3. Calculate Probability Density Function (PDF) $p(r_k) = h(r_k)/MN$.
4. Compute Cumulative Distribution Function (CDF) $T(r_k) = \sum p(r_k)$.
5. Map intensity levels $s_k = \text{round}(255 \cdot T(r_k))$.
6. Display image before and after equalization alongside their histograms.

5.3 Implementation Code Stub

```
1 def custom_histogram_equalization(img_gray: np.ndarray) -> tuple[np.  
   ndarray, np.ndarray, np.ndarray]:  
2     """Computes histogram equalization manually."""  
3     # TODO: Calculate intensity counts for values 0 to 255  
4     # TODO: Compute normalized CDF  
5     # TODO: Scale CDF to [0..255] and round  
6     # TODO: Map input image pixels to new intensities  
7     pass
```

Listing 5: Custom Histogram Equalization Stub

5.4 Discussion & Observations

Compare your manual histogram equalization output with OpenCV `cv.equalizeHist` and analyze contrast distribution...

6 Question 6: Selective Foreground Histogram Equalization

6.1 Task Description

Apply histogram equalization **only to the foreground** of an image (e.g. `q1.jpeg`).

6.2 Steps

1. (a) Split image into H, S, V planes and display each plane in grayscale.
2. (b) Select appropriate plane to threshold and extract binary foreground mask M .
3. (c) Extract foreground using `cv.bitwise_and` and compute foreground histogram.
4. (d) Compute cumulative sum of histogram using `np.cumsum`.
5. (e) Equalize foreground intensities based on cumulative histogram.
6. (f) Recombine background with equalized foreground.

6.3 Implementation Code Stub

```
1 # TODO: Load Fig. 6 image ('q1.jpeg')
2 # TODO: Convert to HSV and display H, S, V planes
3 # TODO: Threshold selected plane to create binary foreground mask
4 # TODO: Apply cv.bitwise_and to extract foreground pixels
5 # TODO: Compute cumulative sum of foreground histogram using np.cumsum()
6 # TODO: Equalize foreground intensities and recombine with background
```

Listing 6: Selective Foreground Equalization Stub

6.4 Discussion & Observations

Discuss plane selection choice for segmentation and compare selective foreground equalization vs global equalization...

7 Question 7: Sobel Filtering & Separable Filtering

7.1 Task Description

Perform Sobel filtering on `q2.jpeg` (Einstein image) using three distinct methods:

1. (a) Using OpenCV `cv.filter2D` with full 2D Sobel kernel.
2. (b) Manual implementation of 2D convolution for Sobel filtering.
3. (c) Utilizing kernel separability property:

$$\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -1 \end{bmatrix} \quad (4)$$

Perform sequential 1D horizontal and vertical convolutions.

7.2 Implementation Code Stub

```
1 def sobel_filter2d_cv(im: np.ndarray) -> np.ndarray:
2     """(a) Sobel filtering using cv.filter2D"""
3     pass
4
5 def sobel_filter2d_manual(im: np.ndarray) -> np.ndarray:
6     """(b) Sobel filtering using manual 2D convolution loop"""
7     pass
8
9 def sobel_filter_separable(im: np.ndarray) -> np.ndarray:
10    """(c) Sobel filtering using 1D separable kernel decomposition"""
11    pass
```

Listing 7: Sobel Filtering Implementations Stub

7.3 Discussion & Observations

Compare execution times and accuracy of 2D vs 1D separable filtering...

8 Question 8: Image Zooming (Nearest-Neighbor & Bilinear)

8.1 Task Description

Write a custom program/function `zoom_image(im, scale_factor, method)` to resize images by factor $s \in (0, 10]$.

8.2 Requirements

1. Implement interpolation methods: (a) Nearest-neighbor interpolation, (b) Bilinear interpolation.
2. Scale up small images by a factor of 4.
3. Compute **Normalized Sum of Squared Differences (SSD)** by comparing up-scaled images against original high-resolution targets:

$$\text{SSD}_{\text{norm}} = \frac{\sum_{i,j} (I_{\text{orig}}(i,j) - I_{\text{zoomed}}(i,j))^2}{\sum_{i,j} I_{\text{orig}}(i,j)^2} \quad (5)$$

8.3 Implementation Code Stub

```
1 def zoom_image(im: np.ndarray, factor: float, method: str = 'bilinear')
  -> np.ndarray:
2     """Zooms image by given scale factor using custom interpolation."""
3     pass
4
5 def compute_normalized_ssd(im_orig: np.ndarray, im_zoomed: np.ndarray)
  -> float:
6     """Computes normalized sum of squared differences."""
7     pass
```

Listing 8: Image Zooming Code Stub

8.4 Discussion & Observations

Compare SSD scores for Nearest Neighbor vs Bilinear interpolation and analyze visual artifacts...

9 Question 9: GrabCut Segmentation & Background Blurring

9.1 Task Description

Consider the Marguerite Daisy flower image.

1. **(a)** Use `cv.grabCut` to segment flower foreground. Show final segmentation mask, extracted foreground image, and extracted background image.
2. **(b)** Produce enhanced image with a substantially blurred background (using Gaussian filter). Display original alongside enhanced image.
3. **(c) Discussion Question:** Why is the background area just beyond the edge of the flower quite dark in the enhanced image?

9.2 Implementation Code Stub

```
1 # TODO: Load flower image
2 # TODO: Define bounding box surrounding flower foreground
3 # TODO: Run cv.grabCut to compute segmentation mask
4 # TODO: Apply Gaussian Blur to background image and composite
```

Listing 9: GrabCut Segmentation Code Stub

9.3 Discussion & Answer to Part (c)

Explain why color bleeding / halo artifacts occur along boundary pixels when applying spatial blurring to background...

10 Question 10: Edge-Preserving Bilateral Filtering

10.1 Task Description

1. (a) Use OpenCV `cv.bilateralFilter` with spatial parameter σ_s and range parameter σ_r . Display original image, Gaussian blurred image, and bilateral filtered image side-by-side.
2. (b) Implement custom bilateral filter from scratch. Compare results with OpenCV's `cv.bilateralFilter`. Evaluate results with a quantitative measure (e.g. MSE, PSNR, or SSIM).

10.2 Implementation Code Stub

```
1 def custom_bilateral_filter(im: np.ndarray, d: int, sigma_color: float,
2   sigma_space: float) -> np.ndarray:
3     """Custom implementation of 2D Bilateral Filter."""
4     pass
```

Listing 10: Custom Bilateral Filter Stub

10.3 Discussion & Observations

Discuss edge preservation behavior, effect of `sigma_color` vs `sigma_space`, and quantitative evaluation metrics...